Final Project: Reinforcement Learning for Motion Planning

Jackie Javier, Pranitha Achanta, Robert McDaniels
The University of New Mexico
CS429

Due 5/8/2026 (Extension to 5/12/2026)

1 Reward Strategies

The reward strategies defined as S1 and S2 in task2.py are as follows:

S1:

- When reaching target: +100
- When encountering obstacle/ out of bounds: -100
- Otherwise: +0

S2:

- When reaching target: +100
- When encountering obstacle/ out of bounds: -100
- When moving closer to target (manhattan distance calculation): +1
- When moving further from target (manhattan distance calculation): -1
- Otherwise: +0

As we can see, S1 is a very simple strategy, while S2 is a slightly more involved evolution of S1. While S1 only provides nonzero feedback when encountering a target/obstacle/out of bounds position, S2 calculates the manhattan distance from the next state to the target position in order to reward any progressive movements made. It should be noted that a positive "progression" is not always the optimal decision, as there could exist a map configuration in which a position is closer to the target but results in a longer path due to obstacle placement.

2 Task 6: Performance Evaluation

To evaluate the performance of SARSA and Q-learning, several experiments were conducted using the provided map abstractions. The evaluation focused on comparing the learning processes under different map complexities, exploration rates, discount values and reward strategies. Episodes counts were tuned according to the smallest approximate episode count in which "test accuracy" stop significantly improving, to a maximum of 5000. We encourage examining the array of generated figures from running the program (/output folder).

The following performance metrics were considered throughout the experiments:

- Training time required for the learning process
- Number of training episodes
- Test accuracy based on all valid initial positions
- Average path length generated by the learned policy

2.1 Complexity of the Map

To push the methods further, we added two additional maps of increasing complexity: zigzag and spiral. All tests use $\gamma, \varepsilon = 0.5$.

On zigzag, SARSA is not able to go above 50% accuracy while Q-learning, because of the confined environment, achieves 98% accuracy.

On spiral, neither technique is able to make any progress and they finish after maximum 5000 episodes with 0.5% accuracy. Interestingly, the S2 strategy actually hurts the performance of Q-learning specifically because the manhattan distance heuristic is almost always in the wrong direction. As a result, it tends to gravitate toward the 4 cardinal directions rather than explore the rest of the spiral.

Map	Algorithm	Time (s)	Accuracy	Episodes	Avg Path Length
1	SARSA	23.12	0.868	5000	34.3
1	Q-Learning	5.47	0.993	1400	34.9
2	SARSA	17.74	0.998	5000	35.8
2	Q-Learning	7.20	0.987	1900	35.5
3	SARSA	21.93	0.861	5000	36.6
3	Q-Learning	14.53	0.947	2400	39.3
4	SARSA	32.47	0.602	5000	30.9
4	Q-Learning	7.59	0.780	1800	34.6
5	SARSA	53.65	0.496	5000	26.2
5	Q-Learning	7.91	0.973	1700	45.1
6	SARSA	108.33	0.005	5000	1.0
6	Q-Learning	115.53	0.003	5000	1.0

Table 1: Complexity comparison across all map abstractions using reward strategy S2

Overall, Q-learning demonstrates better scalability across increasing map complexity. While both algorithms perform similarly on simple environments (Maps 1-2), divergence becomes clear in Map 3-5 where SARSA shows degraded performance due to its on-policy nature, which makes it sensitive to exploratory actions. in contrast, Q-learning benefits from off-policy updates, allowing it to learn optimal policies even when exploration introduces suboptimal trajectories. Map 6 highlights a failure case for both algorithms, where neither method is able to converges effectively under current reward structure (S2), indicating that the environment complexity and sparse feedback is higher than algorithm differences.

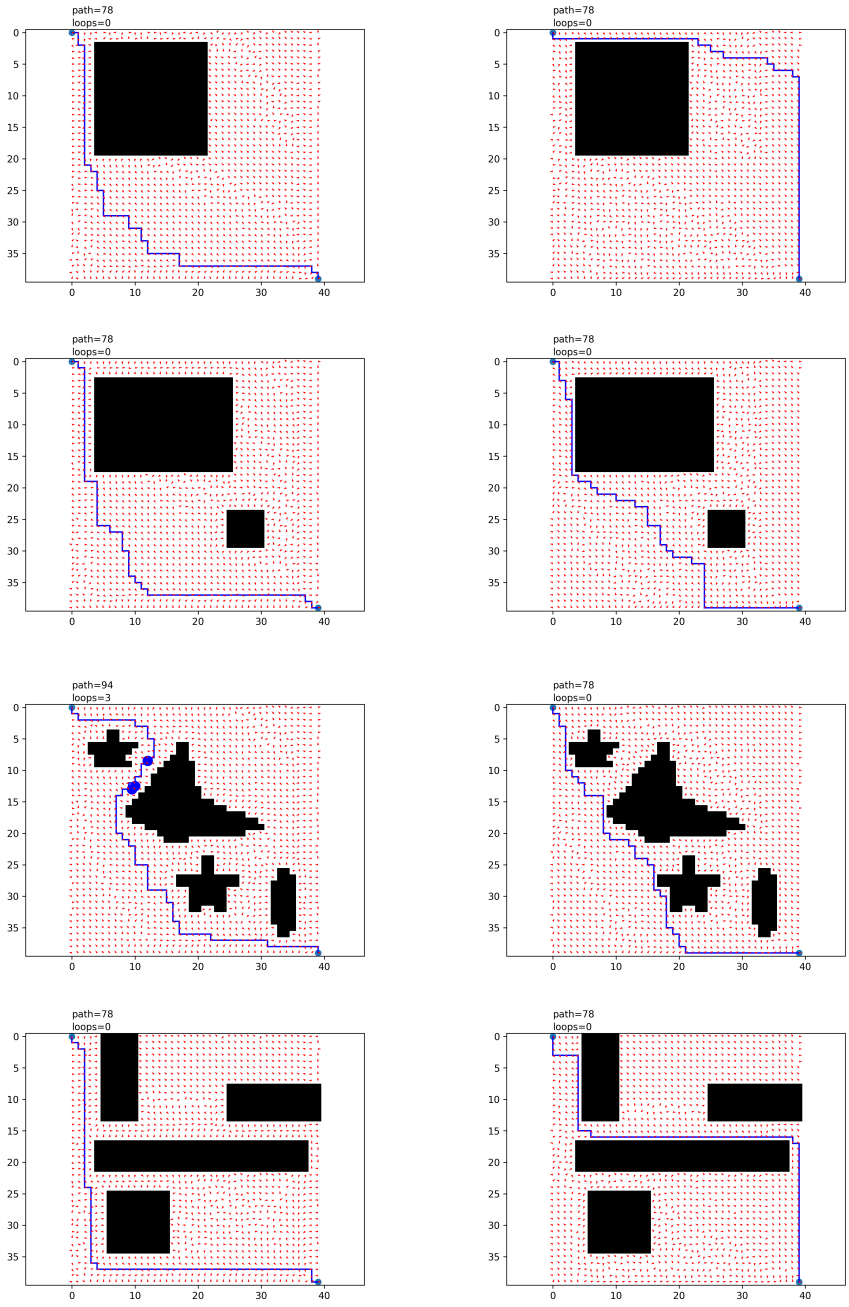

Figure 1: Simple and Medium Complexity Map agent paths from (0,0) to (39,39). Left is SARSA, right is Q-Learning. All are S2.

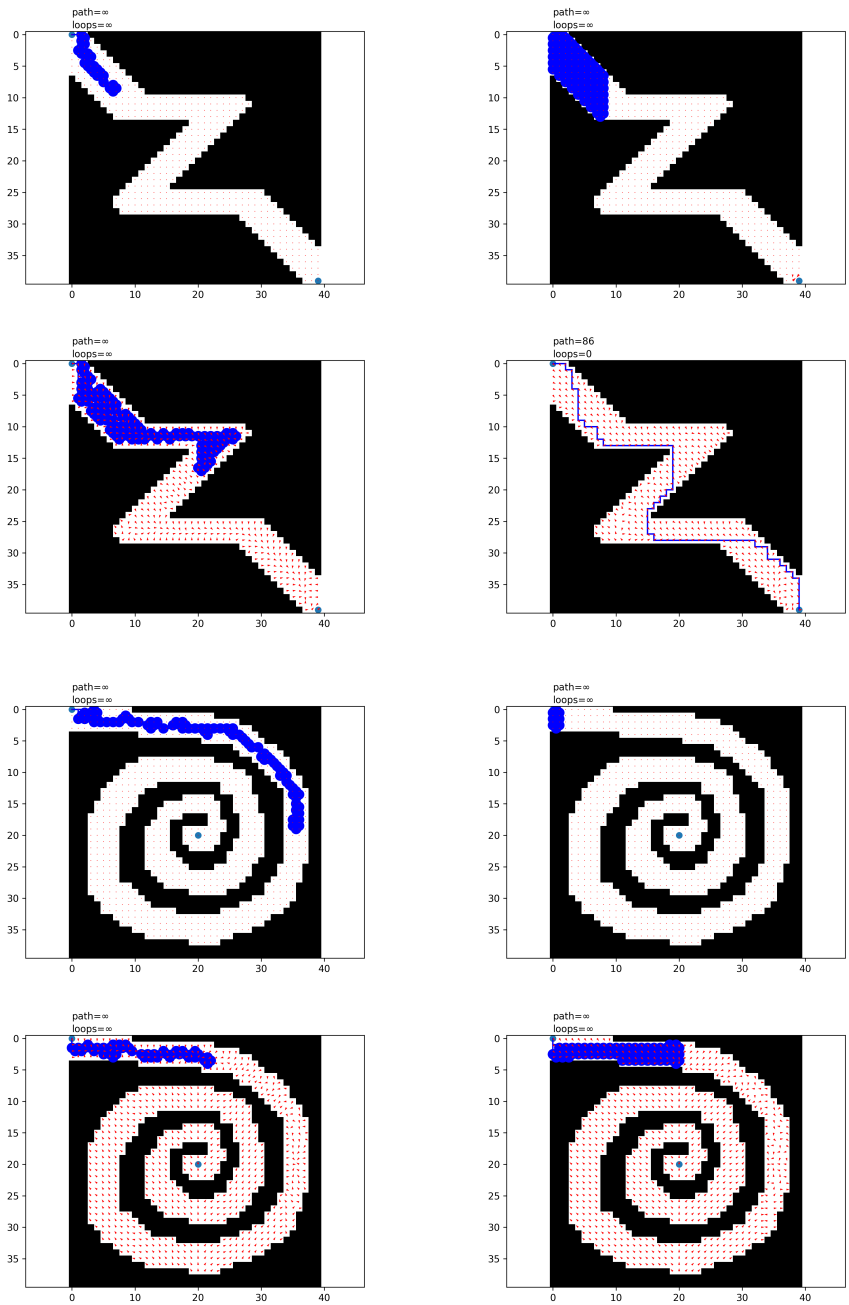

Figure 2: Zigzag and Spiral Map agent paths from (0,0) to (39,39) for both S1 and S2. Left is SARSA, right is Q-Learning. Odd rows are S1, even rows are S2.

2.2 Exploration Rate

We evaluated the effect of exploration probability ε using Map 4 with $\gamma = 0.5$.

ε	Algorithm	Time (s)	Accuracy	Episodes	Avg Path Length
0	SARSA	7.07	0.163	5000	40.6
0.5	SARSA	31.60	0.597	5000	30.9
1	SARSA	74.63	0.555	4900	27.8
0	Q-Learning	8.65	0.170	5000	37.3
0.5	Q-Learning	27.78	0.788	5000	34.7
1	Q-Learning	110.10	0.896	4000	35.7

Table 2: Exploration rate comparison on Map 4

Low exploration ($\varepsilon = 0$) leads to poor performance due to lack of state space coverage. Higher exploration improves learning significantly, especially for Q-learning, which is most successful with $\varepsilon = 1.0$. SARSA improves but remains less stable than Q-learning. Because Q-learning is off-policy, it can effectively "ignore" the randomness of a high exploration rate to update its target policy towards the optimum. This also allows for a faster convergence at 4000 episodes for the 1.0 case. In contrast, SARSA's on-policy nature forces it to reconcile random actions with a stable value function.

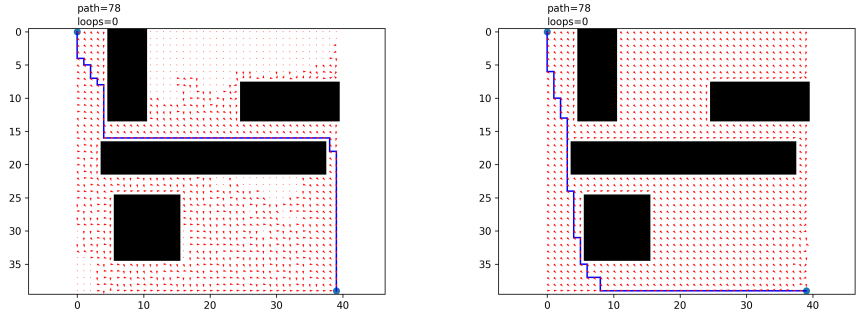

Figure 3: Q $\varepsilon = 0.5$ (left) and $\varepsilon = 1$ (right) on map 4

2.3 Discount Value

We evaluated the different discount factors on Map 4 with $\varepsilon = 0.5$.

γ	Algorithm	Time (s)	Accuracy	Episodes	Avg Path Length
0.1	SARSA	16.80	0.817	5000	36.0
0.5	SARSA	29.16	0.615	5000	31.4
1.0	SARSA	24.41	0.593	5000	33.5
0.1	Q-Learning	7.58	0.770	2000	34.6
0.5	Q-Learning	4.69	0.779	1200	34.6
1.0	Q-Learning	7.89	0.632	2000	35.2

Table 3: Discount factor comparison on Map 4

Lower discount factor ($\gamma = 0.1$) performs best overall, indicating that immediate rewards are important in this environment. SARSA is more important to γ, while Q-learning remains relatively stable across the values. Q-learning achieves competitive accuracy to SARSA in roughly one-fifth of the episodes required by SARSA. This difference in stability is reflected in the figure below, where SARSA's accuracy per 100 training episodes exhibits high volatility compared to the smooth convergence of Q-Learning. As γ increases towards 1.0, the performance degradation suggests that for Map 4, over emphasizing long-term future rewards likely hinders the algorithm's ability to stabilize.

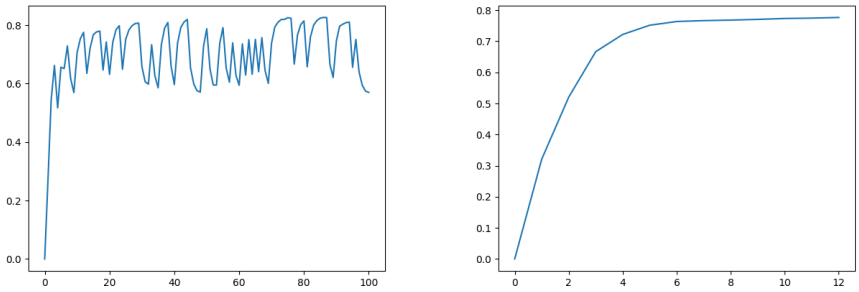

Figure 4: $\varepsilon = 0.1$ for SARSA (left) and Q-Learning (right) on map 4, accuracy per 100 episodes

2.4 Reward Strategy

The following table compares the effect of the two reward strategies on SARSA and Q-learning using the best hyperparameters identified from the previous tests. For SARSA, $\gamma = 0.1$ and $\varepsilon = 0.5$. For Q-Learning, $\gamma = 0.5$ and $\varepsilon = 0.5$

Strategy	Algorithm	Time (s)	Accuracy	Episodes	Avg Path Length
S1	SARSA	77.19	0.033	5000	6.4
S1	Q-Learning	81.65	0.286	5000	52.4
S2	SARSA	81.18	1.000	5000	35.1
S2	Q-Learning	92.27	0.896	5000	37.1

Table 4: Reward Strategy comparison

S2 improves SARSA significantly by providing intermediate reward shaping, which helps to guide learning in sparse environments. A better S1 result would likely require 10000 episodes, rather than our maximum of 5000. However, Q-learning achieves its highest test accuracy under S1, suggesting that dense reward shaping may introduce suboptimal bias in some cases. Since Manhattan distance is a heuristic that does not account for the specific geometry of Map4, it potentially inhibits the potential of Q-Learning's off-policy training algorithm. Overall, S2 improves learning efficiency but does not always guarantee higher final optimality, highlighting the trade-off between reward shaping and convergence accuracy.

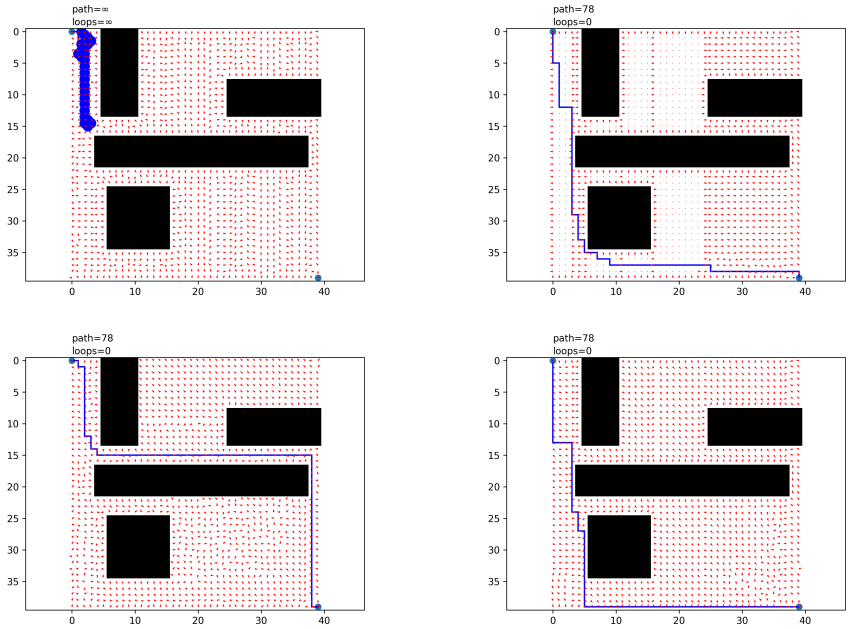

Figure 5: SARSA (left) and Q-Learning (right) on map 4. Row one is S1, row two is S2.

2.5 Additional Test: Policy

As an experiment, we thought to adjust the sampling policy from ε-greedy to softmax multinomial sampling. This introduces a different hyperparameter T (temperature) which adjusts the sharpness of the softmax distribution, with larger T biasing toward exploration and $T = 0$ being equivalent to argmax with random tie-breaking.

The intuition behind this policy is the agent explores actions relative to their value in the Q table in contrast to a flat exploration rate ε. This creates a smooth gradient between exploration and exploitation which adapts as it explores the state-space.

In testing on map4 with 5000 episodes however, it generally performed worse or at best matched the ε-greedy policy. It may simply be that this encourages a kind of "confirmation bias" where it preferentially explores what it already knows.

Method	SARSA Accuracy	Q-Learning Accuracy
Argmax	0.561	0.804
Softmax (0.1)	0.256	0.216
Softmax (0.5)	0.753	0.565
Softmax (1.0)	0.828	0.849
Softmax (1.5)	0.869	0.888
Softmax (2.0)	0.874	0.853

Table 5: Policy sampling comparison (map4, 5000 episodes, $\gamma, \varepsilon = 0.5$)

Softmax policies with moderate temperatures (1.0 - 1.5) perform best. Whereas low temperature leads to poor exploration, while high temperature increases randomness. Q-learning consistently performs better at higher temperature.

3 Team Contributions

- **Jackie**: Developed initial mostly-complete implementations for Tasks 1-6, performed optimizations and corrections across all the tasks, filled tables, and contributed to report writing.
- **Pranitha**: Identified necessary corrections and improvements in Task 1-6 code, contributed to debugging and validation of results, and contributed to report writing.
- **Robert**: Performed optimizations and corrections for Task 1-6, implemented additional map tests and policy evaluations, generated visualizations and contributed to report writing.